

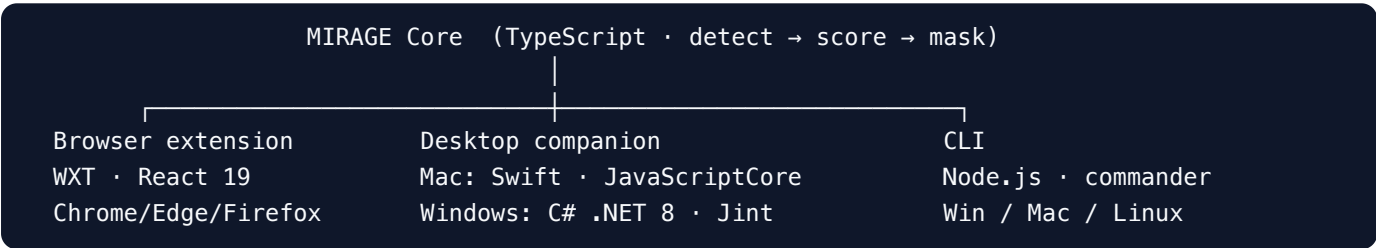
MIRAGE · Tech Stack

What to say when the judges ask "What's your tech stack?"

Say this (30 seconds)

"MIRAGE is **one detection engine in TypeScript**, and we run the **same engine in three products**. The **browser extension** is built with **WXT and React** on Manifest V3. The **desktop companion** is native: **Swift and SwiftUI** on Mac, using Apple's **Accessibility API** to read the ChatGPT app's message box, and **C# .NET** on Windows with **UI Automation**. Both run our engine inside a built-in JavaScript engine. The **CLI** is **Node.js** for developers. Detection is ours: **checksums** like Verhoeff for Aadhaar and Luhn for cards, **entropy** for API keys, and **context rules**, which give a risk score from 0 to 100. Real values are stored **AES-GCM encrypted** on the device for 24 hours. There is **no server**: nothing leaves the device except the protected prompt."

One engine, three products



The stack

PART	TECHNOLOGY	WHY
Detection engine	TypeScript (strict), zero dependencies, bundled with Vite into one file	Write once, run everywhere; the same code gives the same results in every product
How it detects	Text clean-up (hidden characters, look-alike letters, Indian digits) → regex + checksums (Verhoeff, Luhn, PAN rules) → Shannon entropy → context words → risk score	Checksums mean random numbers aren't flagged; no AI model needed, so it's fast and private
Browser extension	WXT (Manifest V3), React 19, Shadow DOM for the UI in chat pages	Works on ChatGPT, Gemini, Claude, Copilot, Perplexity; the chat page can't restyle or read MIRAGE's UI
Desktop, Mac	Swift 6, SwiftUI + AppKit, Accessibility API, CGEventTap, JavaScriptCore	Native app; reads only the AI app's message box; holds Enter until you decide
Desktop, Windows	C# .NET 8, WinForms, UI Automation, Jint (JavaScript engine for .NET)	The same engine on Windows; installed with one .bat file
CLI (Ninad)	Node.js 20, TypeScript, commander, tsup	For developers who paste logs and code into AI; works with Claude Code, Codex and the clipboard
Security	AES-GCM encryption (Web Crypto; Node crypto for the CLI), values expire after 24 hours, secrets never stored	Real values stay on the device and are encrypted at rest

Testing and CI	Vitest, Puppeteer end-to-end tests, a 181-prompt evaluation set, GitHub Actions	F1 0.998; 0.05 ms per prompt; the Windows build is tested on Windows automatically
Installers	Inno Setup (Windows .exe), .bat installer with a SHA-256 check, GitHub Releases	Friends can install with one click

If they ask more

QUESTION	ANSWER
Why not use an AI model to detect?	Speed and privacy. Rules with checksums run in 0.05 ms on the device, need no internet, and every result can be explained ("Why 100?"). A model would need a server or a big download.
How do you run one engine on Mac, Windows and the browser?	The engine is plain TypeScript bundled into one JavaScript file. The browser runs it directly, the Mac runs it in JavaScriptCore, Windows in Jint, and the CLI in Node.
How does the desktop app see ChatGPT's text?	Through the operating system's accessibility interface (the same one screen readers use), and only for the message box of ChatGPT and Claude. No keylogger, no screen recording.
Is there a backend?	No. No server, no account, no database. Only the protected prompt goes to the AI.
What's encrypted?	The map from placeholder to real value (e.g. «PAN_1» → the real PAN), AES-GCM, on the device, deleted after 24 hours. Secrets like API keys are never stored at all.